

THE 60 PERCENT OF THE JOB NOBODY DEMOS

Data Cleaning for AI Engineers

Models are commodities. The cleaning decisions underneath them are what actually separate a working system from a demo.

Cleaning Is The Job

FRAMING

You cannot fix a data problem with a model. Every hour spent on the input is worth several spent tuning what sits on top of it.

60%

of project time is data
work

80%

of failures trace back to
data

0

models that fix bad labels

WHAT GETS ATTENTION

- Model architecture
- Hyperparameters
- Prompt wording
- GPU budget

WHAT DECIDES THE OUTCOME

- Label quality
- Missing value handling
- Leakage
- Duplicate and near duplicate rows



Cleaning is unglamorous
and decisive



And almost never shown in a
demo

If your model suddenly performs brilliantly, look for leakage before you celebrate. It is the correct first hypothesis.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

The Cleaning Pipeline

ORDER

Order matters. Splitting before you clean is the difference between an honest score and a fantasy.

Profile

,

Split

,

Fix
types

,

Handle
missing

,

Dedupe

,

Validate

STEP

WHY IT SITS HERE

Profile first

You cannot fix what you have not measured

Split second

Everything after this must be fit on train only

Types before values

A string column hides every numeric problem

Validate last

A schema check that runs on every future batch



Fit on train, transform
everything



Every step must rerun on
new data

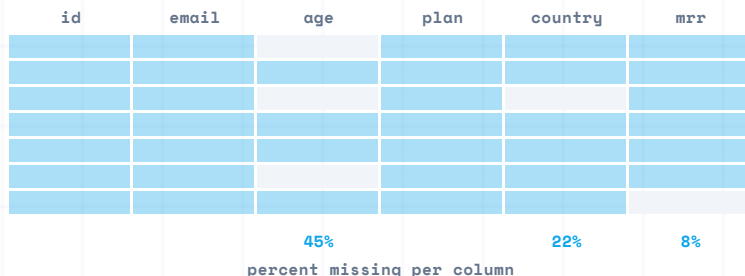
Fitting an imputer or a scaler before the split leaks test information into training, and it inflates every number you report.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Look Before You Touch

PROFILING

Ten minutes of profiling saves days. You are looking for the shape of the mess, not for the mess to be gone.



plot this before you write a single transform

```
df.info(); df.describe(include='all')
df.isna().mean().sort_values(ascending=False)
df.nunique(); df.duplicated().sum()
```

WHAT YOU ARE LOOKING FOR



Columns more than half empty



Ranges that are physically impossible



Categories that are the same thing twice



Dates in the future or before the company existed

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Missing Values

STRATEGY

The right fix depends on why the value is missing, and that is a question about the process that produced the data, not about the column.

PATTERN	MEANING	APPROACH
MCAR	Missing at random, no pattern	Impute or drop, low risk
MAR	Explained by other columns	Model based imputation
MNAR	Missing because of its own value	Add a flag, never silently fill

```
# always keep the fact that it was missing
df['income_missing'] = df['income'].isna().astype(int)
df['income'] = df['income'].fillna(train_median) # train only
```



Add a was_missing flag before you fill anything



Never fill with zero unless zero is meaningful

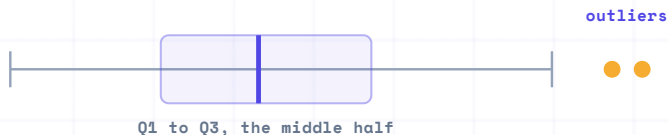
A missing income on a loan form is not random. The missingness is often the strongest feature in the dataset.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Outliers

JUDGEMENT

Three ways to find them, one rule for what to do next: understand it before you remove it.



IQR rule: beyond $Q1 - 1.5 \cdot IQR$ or $Q3 + 1.5 \cdot IQR$

an outlier is a question, not automatically a deletion

METHOD	BEST FOR
IQR rule	Skewed data, the safe default
Z score beyond 3	Roughly normal distributions only
Isolation Forest	Multivariate, odd combinations of normal values
Domain caps	When you know the physical limit

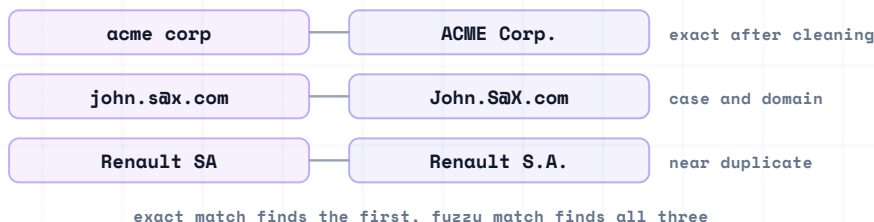
An impossible value is a data error and should go. An extreme but real value is signal, and deleting it teaches the model that reality does not happen.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Exact and Near

DEDUPE

Duplicates inflate your metrics twice: once by giving the model repeated examples, and once by putting the same row in train and test.



```
df = df.drop_duplicates(subset=['email'], keep='last')  
# near duplicates need a similarity threshold  
from rapidfuzz import fuzz  
fuzz.token_sort_ratio(a, b) > 92
```



Dedupe on a business key,
not the whole row



Keep the most recent, not
the first seen

Deduplicate before splitting, never after. A duplicate on both sides of the split makes your test set partly a training set.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Types and Formats

NORMALISE

Most dirty data is not wrong, it is inconsistent. The same fact written four ways is four facts as far as the model is concerned.

```
df['date'] = pd.to_datetime(df['date'], errors='coerce', utc=True)
df['amount'] = (df['amount'].astype(str)
                .str.replace(r'[$,]', '', regex=True)
                .pipe(pd.to_numeric, errors='coerce'))
df['country'] = df['country'].str.strip().str.upper()
```

SYMPTOM	USUAL CAUSE
A numeric column typed object	One stray string or currency symbol
Dates off by hours	Naive timestamps and timezone drift
Duplicate categories	Trailing spaces and mixed case

`errors="coerce"` turns unparseable values into NaN instead of crashing. Always count how many it created.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Text for LLM Pipelines

CHUNKING

Text going into an embedding model needs different cleaning from text going into a classifier. Over cleaning is the bigger risk here.

```
import re, unicodedata
t = unicodedata.normalize('NFKC', t)           # unify unicode
t = re.sub(r'<[>]+>', ' ', t)                 # strip markup
t = re.sub(r'\s+', ' ', t).strip()             # collapse space
```



Do not lowercase text
headed for an embedding
model



Keep punctuation, it carries
sentence boundaries



Keep the raw text for
citation and display



Extract tables separately,
they never survive flattening

Cleaning for a classifier and cleaning for an embedding model are different jobs. Stripping case and punctuation helps one and hurts the other.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

High Cardinality

LONG TAIL

A category column with 4,000 values becomes 4,000 columns and almost no signal. Cut the tail before encoding.



CARDINALITY	ENCODING
Under 15 values	One hot, simple and safe
15 to 100	Target encoding, fit inside the CV fold
Over 100	Group rare into other, or learn an embedding



Reserve an explicit unknown bucket



Freeze the category list with the model

Whatever you do to the training categories, unseen values will appear in production. Decide now what happens to them.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Scaling and Skew

TRANSFORM

Distance based models and gradient descent both assume comparable ranges. A column in rupees and a column in years are not comparable.

TRANSFORM	USE WHEN
StandardScaler	Roughly normal, the default for linear and neural models
MinMaxScaler	You need a bounded range, images and some nets
RobustScaler	Outliers are present and real
log1p	Heavy right skew: money, counts, durations
None	Tree models, they do not care about scale

```
# fit on train, apply to both, always
sc.fit(X_train); X_train = sc.transform(X_train)
```

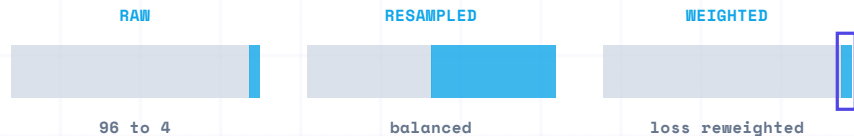
Scaling changes nothing for tree models and everything for KNN, SVM and neural networks. Know which one you are feeding.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Class Imbalance

SKEW

When 4 percent of rows are the positive class, accuracy stops meaning anything and your default threshold stops working.



accuracy on the raw split is 96 percent for a model that never predicts the minority

APPROACH	TRADE OFF
Class weights	No data change, first thing to try
Undersample majority	Fast, throws away real data
SMOTE and friends	Synthetic rows, easy to fool yourself with
Change the threshold	Often the whole fix, and it costs nothing



Try the threshold before you touch the data



Report PR AUC, not ROC AUC, on rare classes

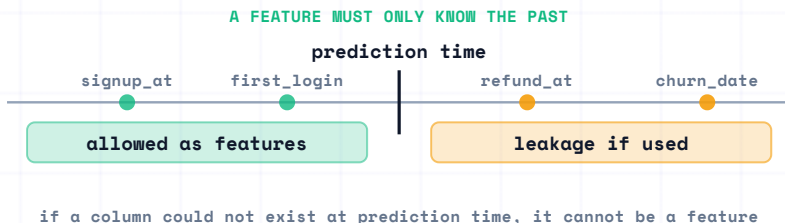
Never resample the test set. It must reflect the real world, or your numbers describe a world that does not exist.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

The Silent Killer

LEAKAGE

Leakage is any information in your features that would not exist at prediction time. It produces excellent offline scores and a model that fails on day one.



SOURCE	EXAMPLE
Target leakage	A column filled in only after the outcome
Train test contamination	Scaling or imputing before the split
Temporal leakage	Random split on time ordered data
Group leakage	The same user on both sides of the split

The test: for every feature, ask when this value is written. If the answer is after the thing you are predicting, it is leakage.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Label Quality

GROUND TRUTH

Your ceiling is the quality of your labels. A model cannot be more consistent than the humans who taught it.



Measure inter annotator agreement before trusting a set



Write a labelling guide with real edge cases in it



Double label a sample and review every disagreement



Inspect the rows your model gets most confidently wrong

SIGNAL	WHAT IT TELLS YOU
Low agreement	The task definition is ambiguous, not the model
Confident errors	Mislabelled rows, look there first
One annotator drifting	A training or guideline problem

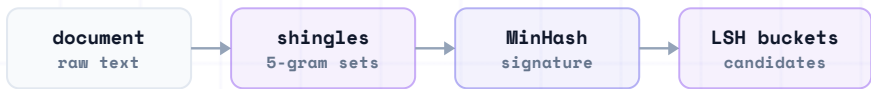
If two annotators agree only 70 percent of the time, no model will exceed roughly 70 percent. Fix the guideline, not the architecture.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Deduplicating a Corpus

MINHASH

Training or embedding a large corpus means comparing every document to every other one, which does not scale. MinHash plus LSH makes it tractable.



compare only inside a bucket, not all pairs

n^2 squared comparisons become roughly linear

how you dedupe a training corpus of ten million documents



Duplicated documents get memorised, not learned



They also silently reweight your data distribution



Typical thresholds sit around 0.8 Jaccard similarity



Dedupe before chunking, or you dedupe the same text many times

Benchmark contamination is this problem in disguise: the test set leaked into the training corpus as a near duplicate.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

PII and Redaction

COMPLIANCE

Cleaning is also the point where personal data should stop travelling. Once it is in a vendor log, it is out of your control.

```
import re
PATTERNS = {
    'email': r'[\w.+-]+@[\w-]+\.[\w.]+',
    'phone': r'\+?\d{1,3}\d{3,4}',
    'card': r'\b(?:\d{16}|\d{15})\b',
}
```



Run NER for names, regex for structured ids



Keep a reversible token if you need to join back later



Redact before logging, not before displaying



Test the redactor on real formats, not toy strings

Regex is precise and blind. NER is broad and fuzzy. Production redaction needs both, with regex running first.

↓ [CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Validation

CONTRACTS

Cleaning is not a one off script. It is a contract that every future batch has to satisfy, checked automatically.

```
from pydantic import BaseModel, Field

class Row(BaseModel):
    email: str = Field(pattern=r'[^.]+@.+.+')
    age: int = Field(ge=0, le=120)
    mrr: float = Field(ge=0)
```

CHECK	CATCHES
Schema and types	An upstream field that changed shape
Ranges	A unit change, cents becoming dollars
Null rate	A broken join or a dead integration
Row count drift	A partial load nobody noticed

Run the contract on every batch, not once at the start. Data quality is not a state you reach, it is a check you keep running.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Make It Reproducible

VERSIONING

A cleaning step you cannot rerun is a result you cannot defend.
Treat the pipeline like code, because it is code.



Cleaning lives in version control, not a notebook cell



Fix and log every random seed



Version the dataset alongside the code that made it



Record what was dropped and why, with counts



Every transform is a fitted object, saved with the model



A test asserts the row count after each step

The question that ends most debates: can you regenerate this exact dataset from raw, today, with one command.

If the answer is no, the result is not reproducible, whatever the notebook says.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

The Checklist

BEFORE TRAINING

Run through this before you train anything you intend to report on.

CHECK	PASS CONDITION
Split done first	Nothing was fit before the split
Duplicates removed	Deduped before splitting, not after
Missing understood	Every column has a documented decision
Leakage audit	Every feature could exist at prediction time
Labels sampled	A human reviewed 100 rows by hand
Distributions logged	You can compare against production later

IF YOU ONLY HAVE TIME FOR TWO

 Audit for leakage

 Read 100 rows by hand

Reading a hundred raw rows yourself finds problems no summary statistic will ever show you.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

The Cleaning Toolkit

 Profiling Split first Missing values Outliers Deduplication Text normalisation High cardinality Imbalance Leakage PII redaction Validation contracts Reproducibility

**Anyone can call fit and predict.
Engineers know what the data
went through first.**

Save this before your next ML project.